



Agents II

June 8, 2026

Recap: LLM Agents

Agents:

LLMs with access to tools and the freedom to use them to get things done.



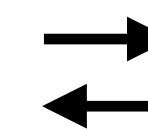
“What is the weather like today in Bochum?”



“I don’t know.”



“What is the weather like today in Bochum?”



Google

“22°C”



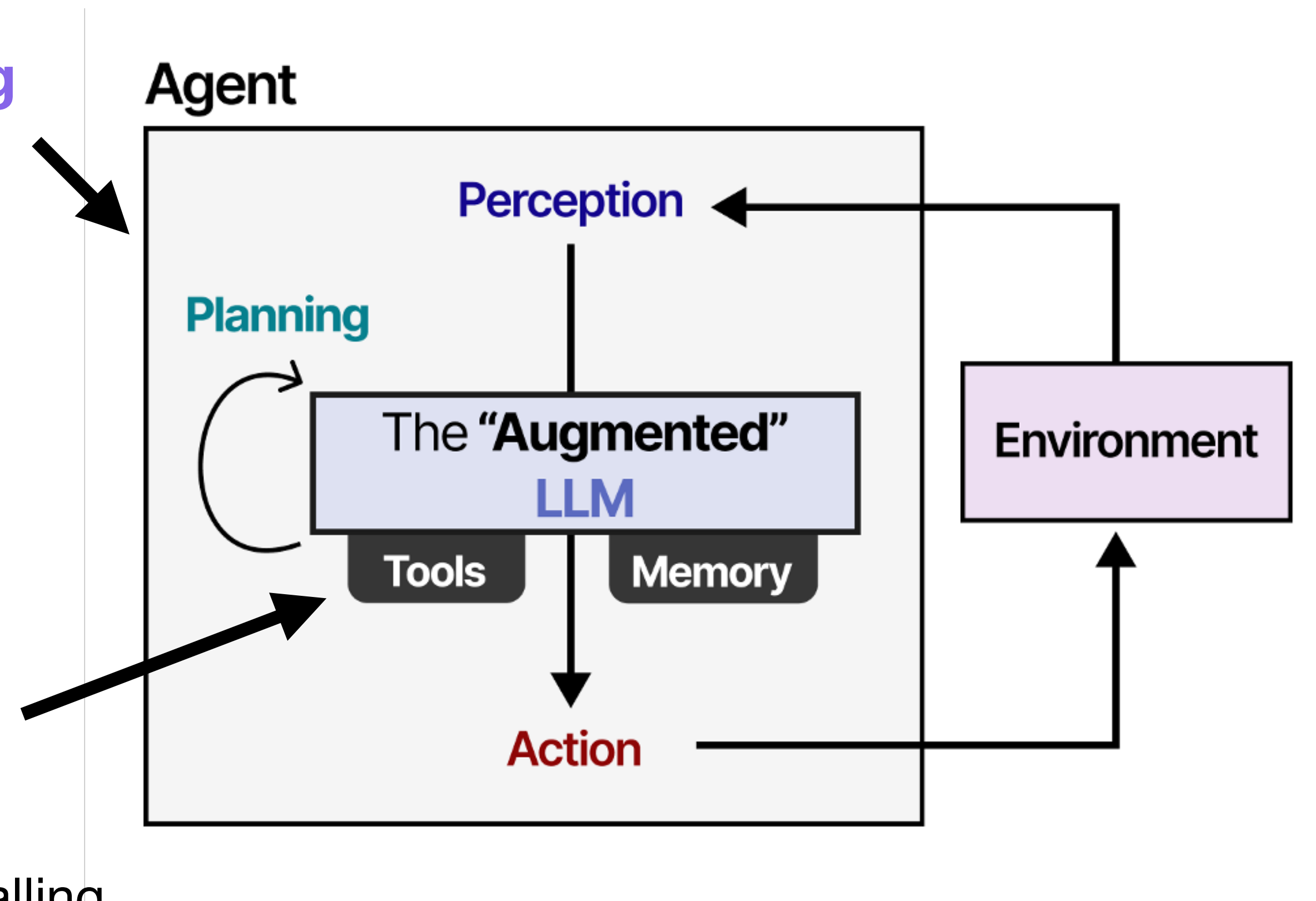
Recap: Agent Loop

Planning through Reasoning

- Chain-of-Thought
- Decomposition of problems
- Reflection
- ...

Tools

- Search
- API
- Function calling
- ...



Recap: Tools

Kinds of Tools

	Example	Why it matters
Information tools	search, RAG, database lookup	give the model external knowledge
Computation tools	calculator, Python, test runner	give exact or executable feedback
Action tools	email, browser, calendar, file edits	let the agent change the world



Recap: Tool Design

Tool use requires a contract between model and system.

Includes:

- Tool name
- Natural-language description
- Input schema
- Output schema
- Error behavior
- Permissions
- Examples



2nd half today: How to expose & discover tools

Recap: Specifying Tools

System Prompt:

You are Qwen, created by Alibaba Cloud. You are a helpful assistant.

Tools (If Specified):

You may call one or more functions to assist with the user query.

You are provided with function signatures within `<tools></tools>`

XML tags:

```
<tools>
  for tool in tools
    {{- "\n" }}
    {{- tool | tojson }}
  endfor
</tools>
```



```
<tools>
{"type": "function",
 "function":
 {"name": "calculator", "description": "...",
  "parameters": {...}}}
</tools>
```

For each function call, return a json object with function name and arguments within `<tool_call></tool_call>` XML tags:

```
<tool_call>
  {\ "name\ ": <function-name>, \ "arguments\ ": <args-json-object>}
</tool_call>
```

User Messages:

...

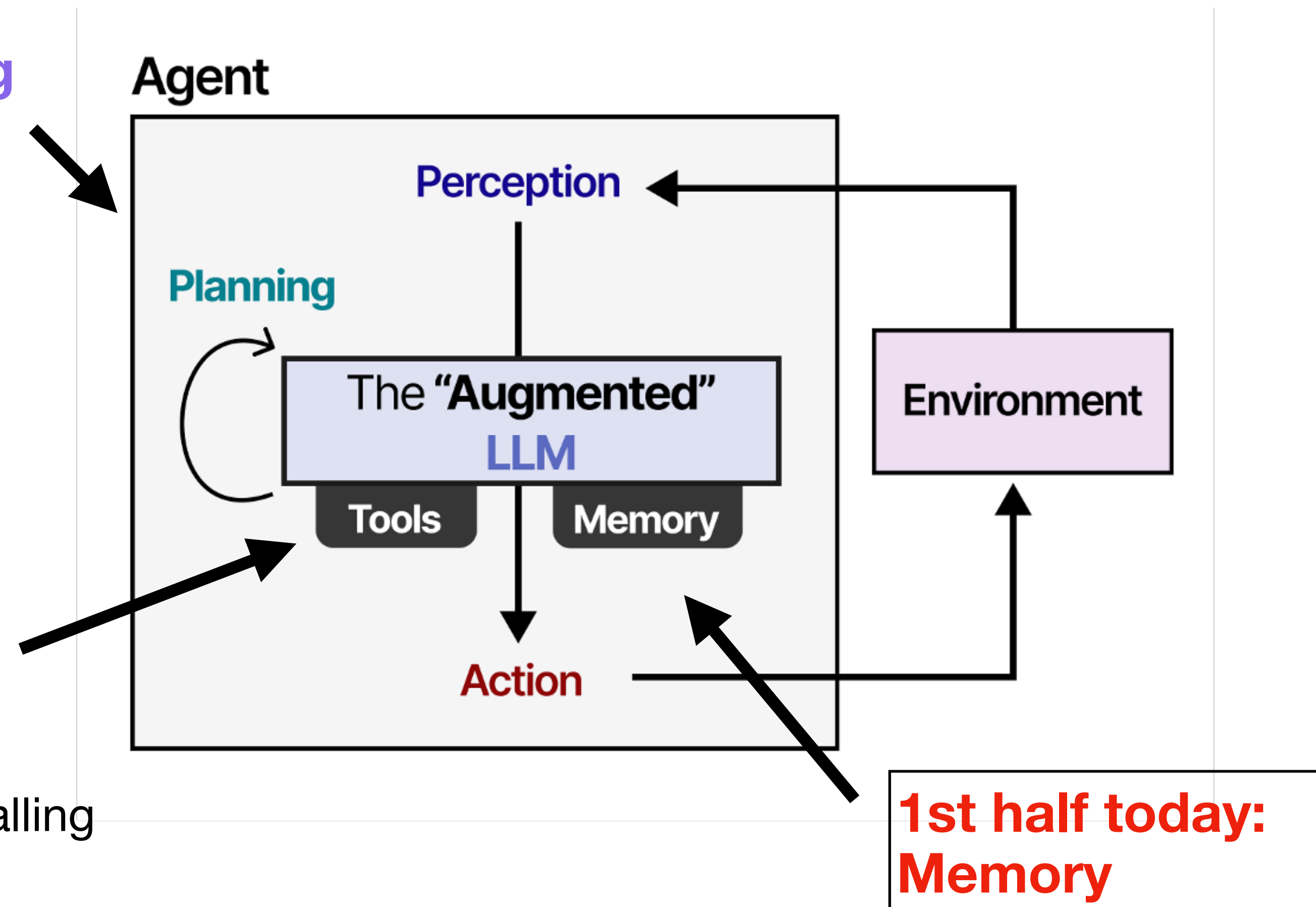
Recap: Agent Loop

Planning through Reasoning

- Chain-of-Thought
- Decomposition of problems
- Reflection
- ...

Tools

- Search
- API
- Function calling
- ...



First Half:

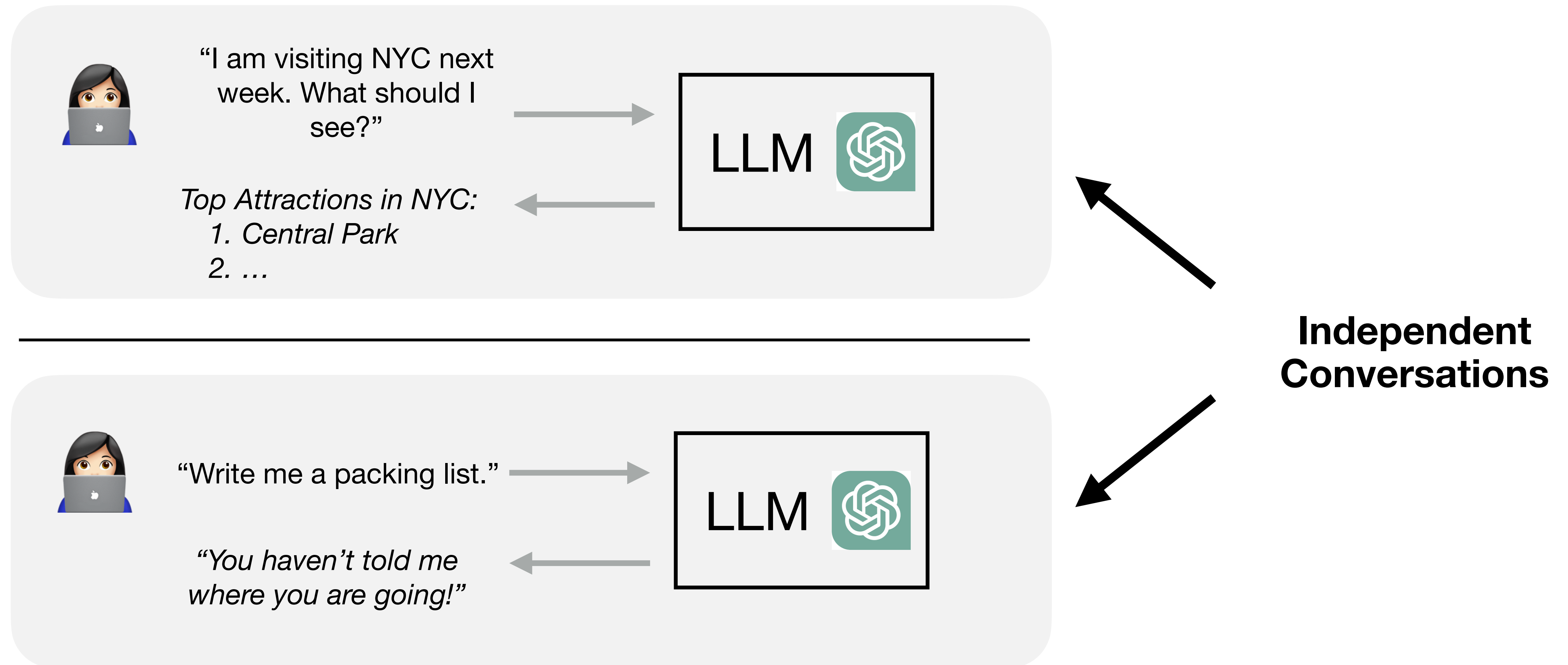
LLM Agent Components & Architectures

Keeping State

	Description	Coding Agent Example
Context	What the model sees in the current call	prompt, current file, last tool call, last test output
State	What the system tracks for the current task	current plan, completed steps (files already edited)
Memory	What persists beyond across tasks	user preferences, past actions, learned facts

Models need to keep track of a lot of information

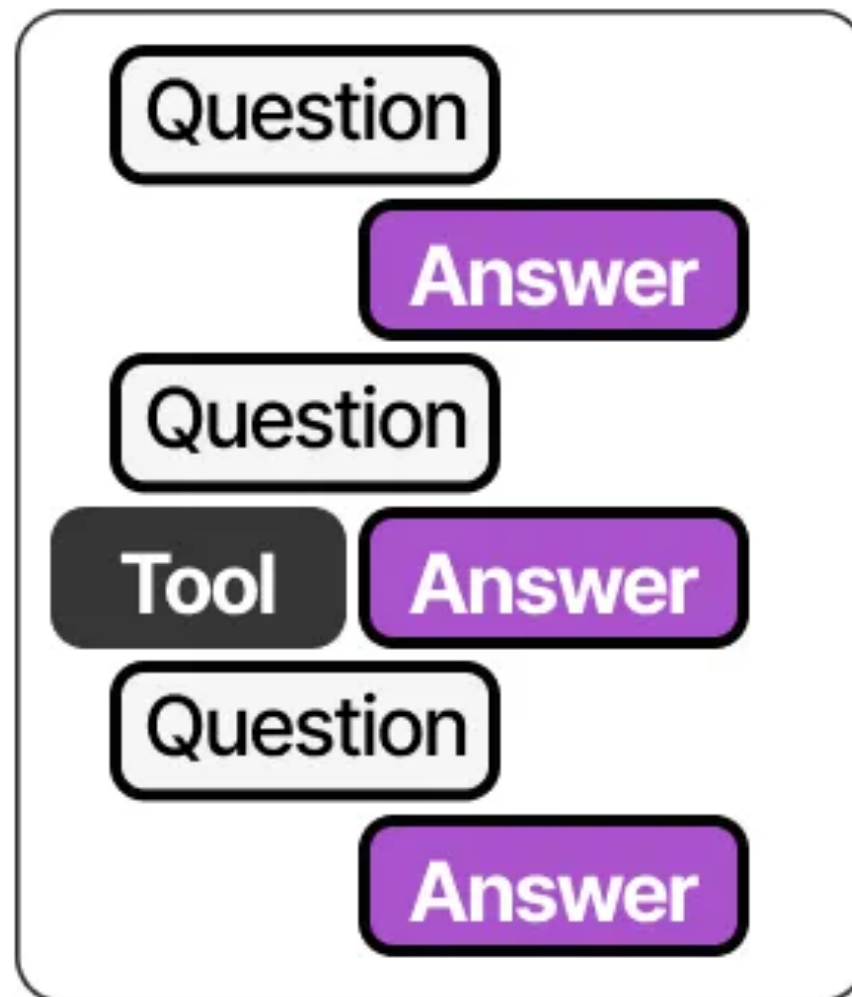
Memory



Potentially hundreds of conversations and steps!

How Fast Does Context Grow?

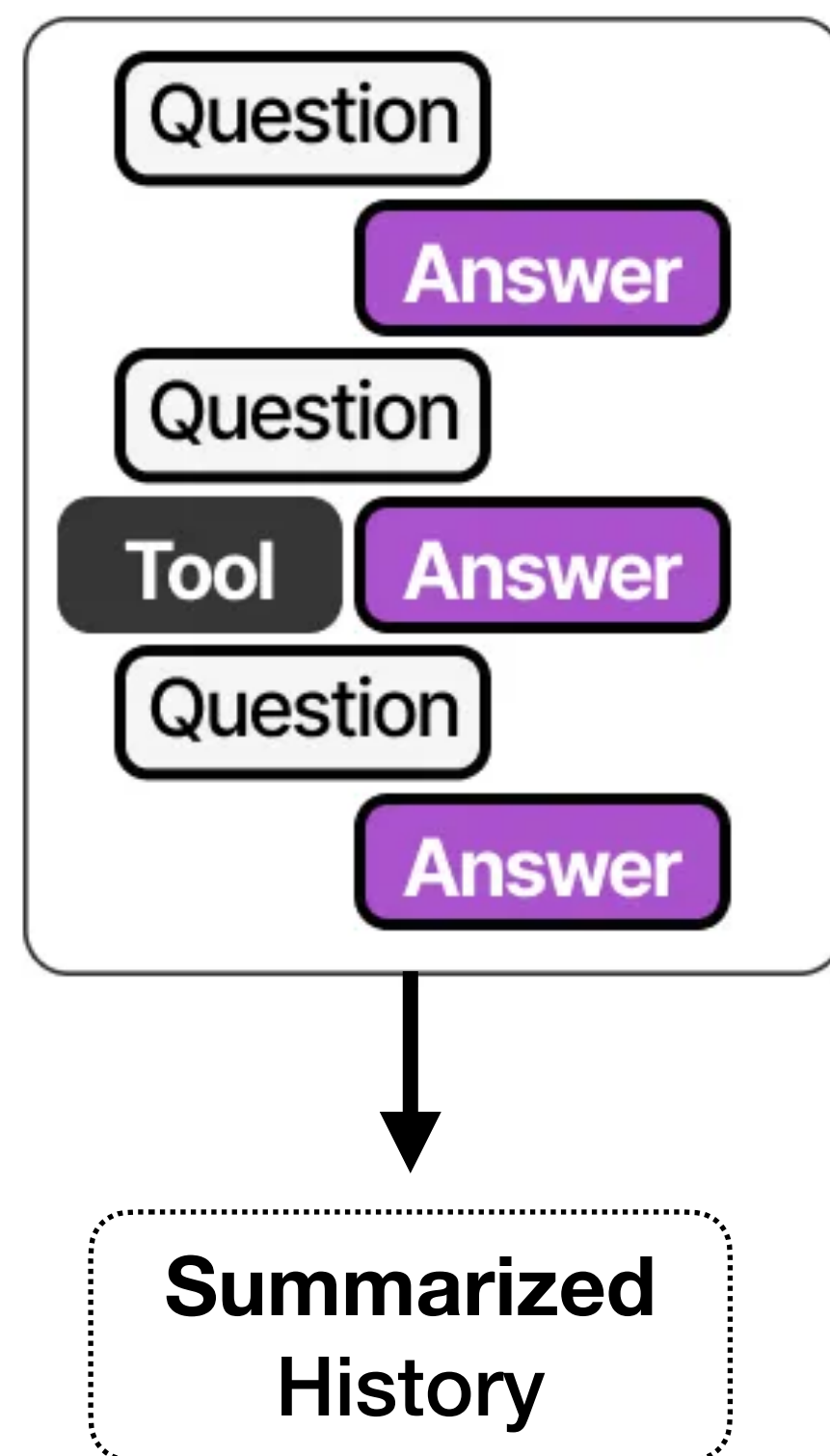
Example: Coding Agent solving a medium bug



Maximum context length:
8,192 tokens

- System prompt + tool schemas:
~800 tokens
- Per iteration:
~300 tokens (model reasoning)
+ ~600 tokens (tool output)
- After 10 iterations:
→ $800 + 10 \times 900 = 9,800$ tokens
- Real coding agents run 20 - 50 iterations
→ 18,000–45,000 tokens

Short-Term Memory

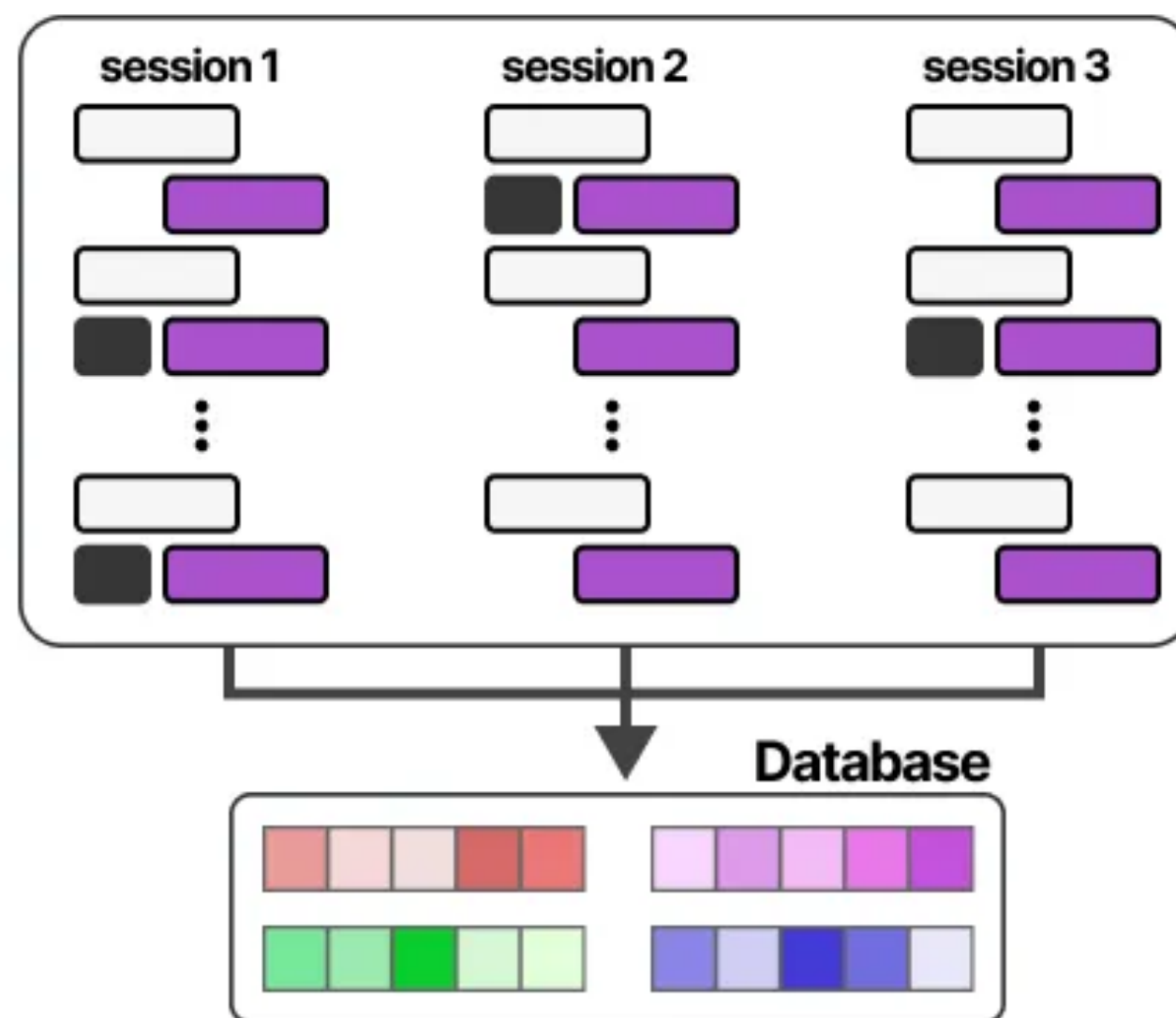


Solution: Continuously summarize conversations!
(e.g., via another LLM)

Multiple Strategies

- Sliding window
- Summarization
- Selective retention
- Memory write tool

Long-Term Memory

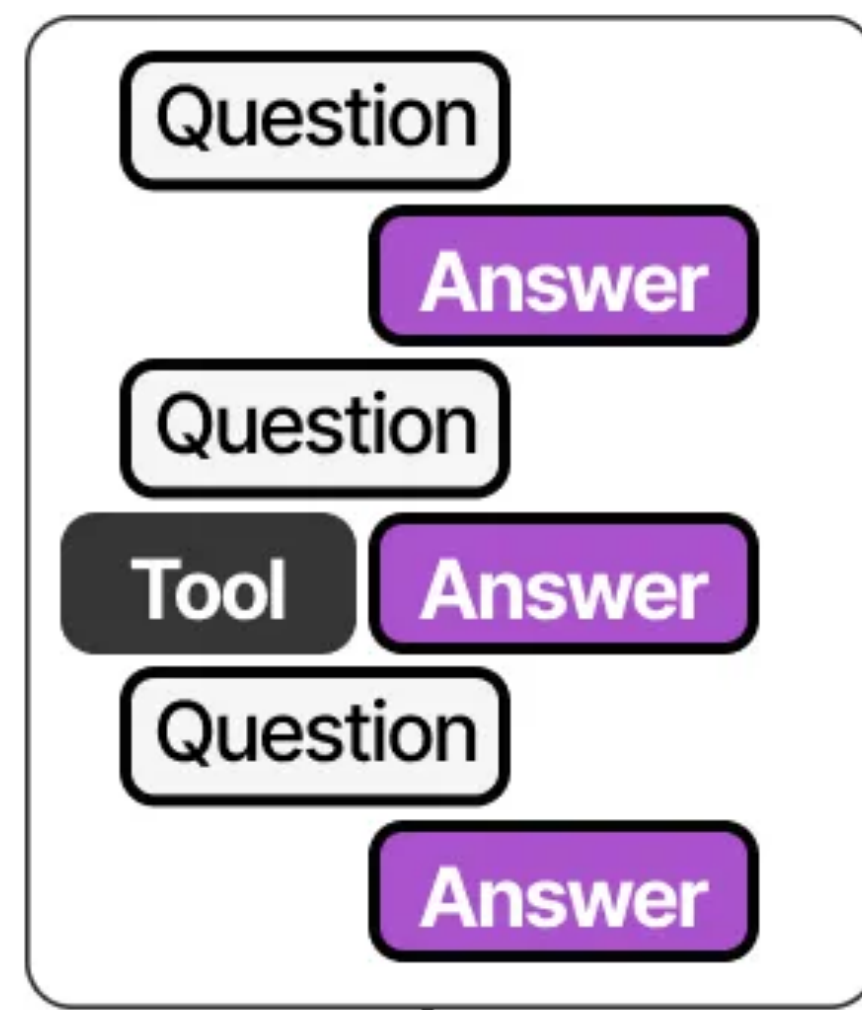


Store previous interactions, actions, and conversations in **external vector database (RAG)**

1. **Build vector database:**
Embed conversations into numerical representations
2. **Embed user query**
3. **Search related** information in the database
4. **Extract most relevant** information from database

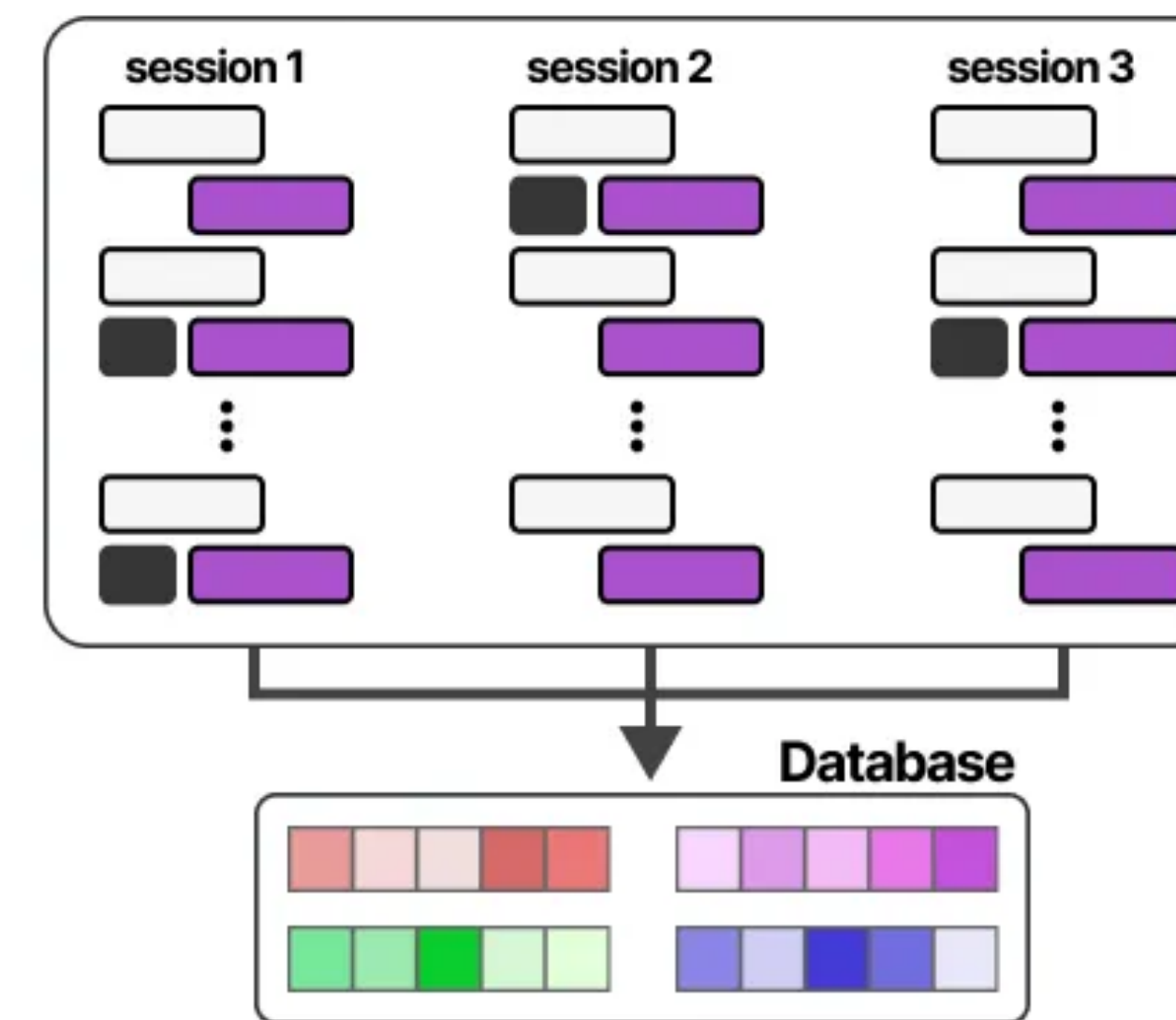
Keeping State: Short-Term vs. Long-Term

Short-Term Memory



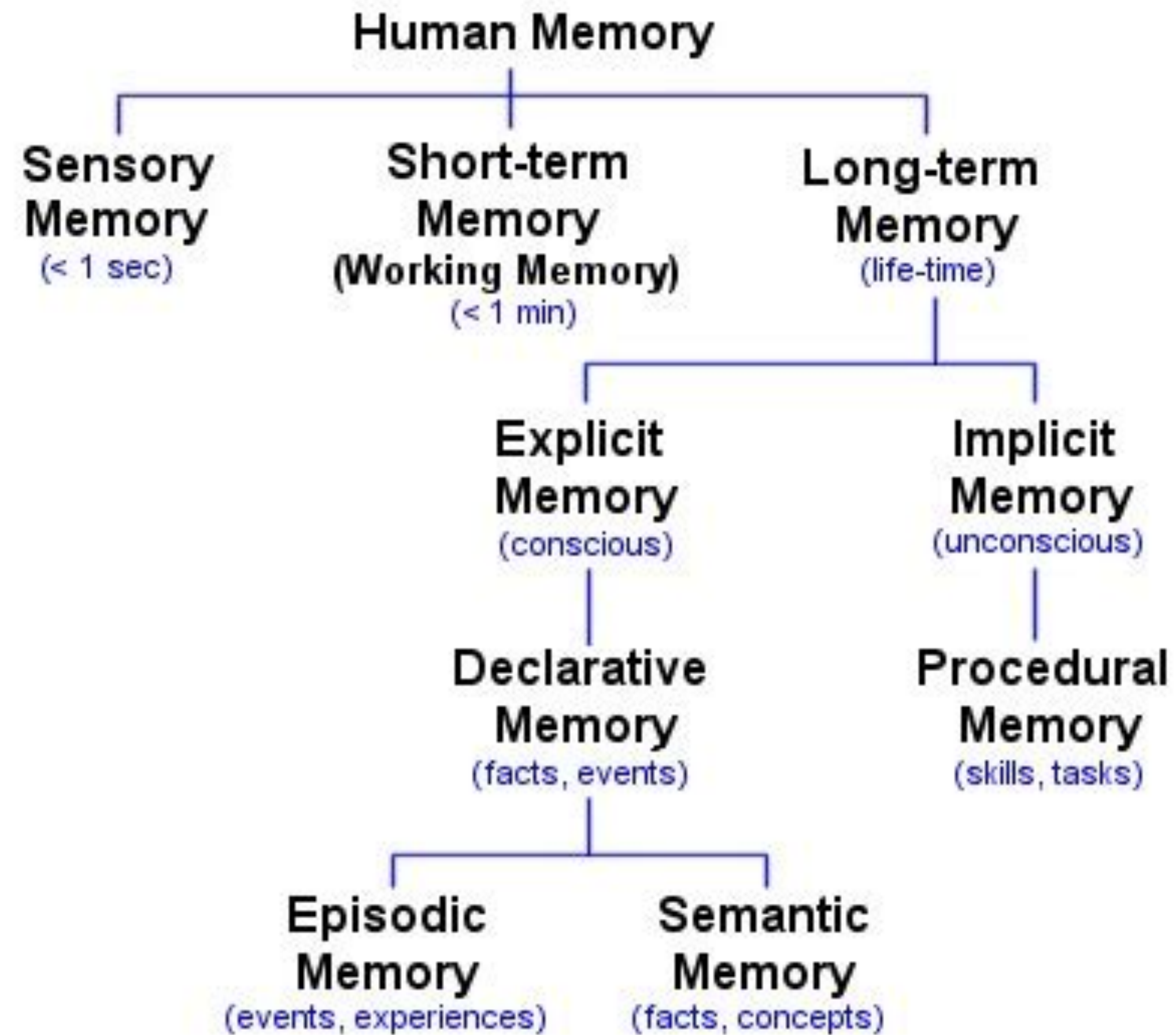
- **Recent** conversations and actions
- Using the model's context window

Long-Term Memory



- Conversations and actions over an **extended period** or **across sessions**

Human Memory Taxonomy

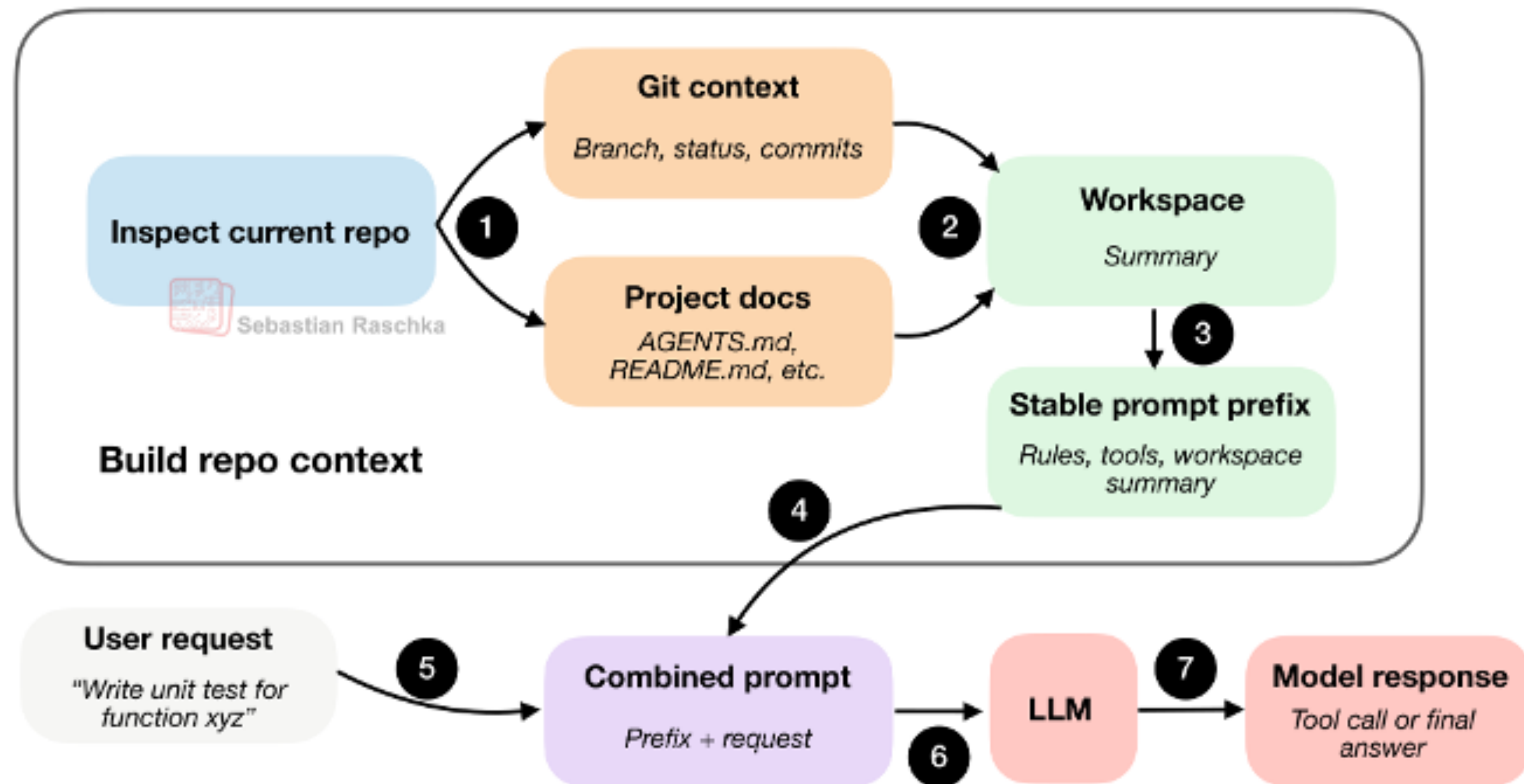


Multiple types of memory exist in psychology

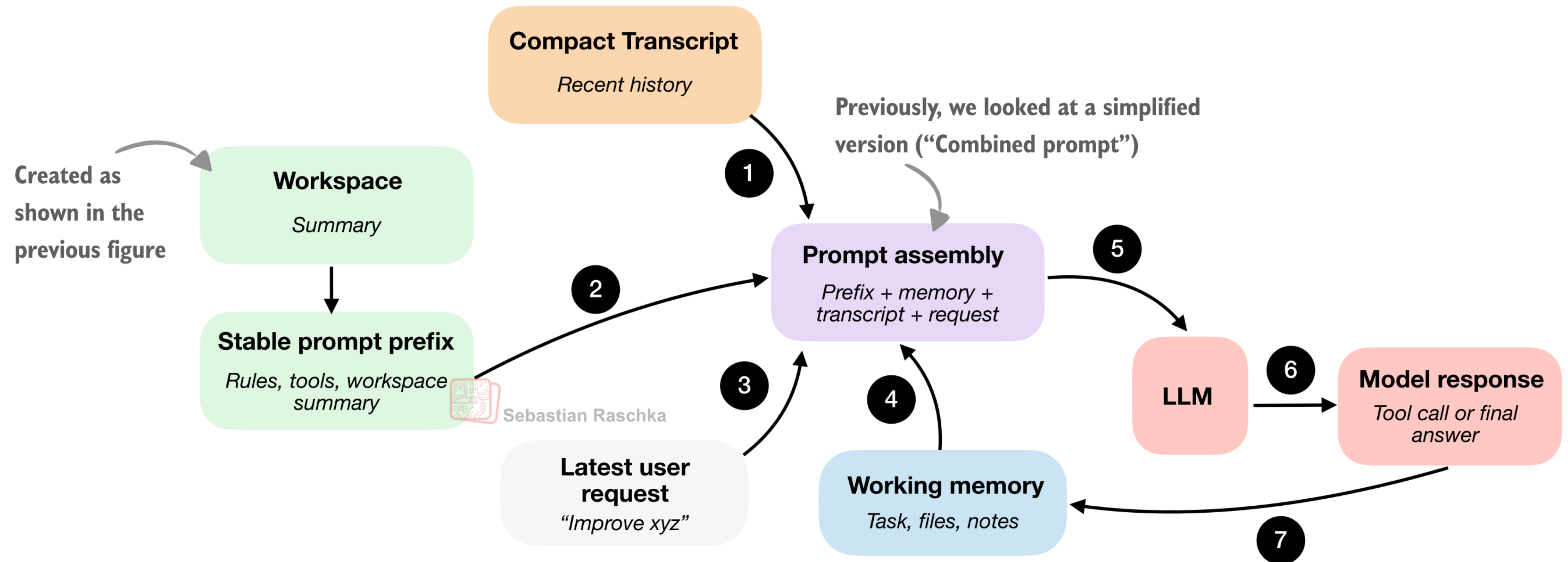
Agent Memory

	Description	Example
Working memory	Current task context	current plan, observations
Procedural memory	How the agent behaves	system prompt, policies, skills
Semantic memory	Facts about world/user	user preferences, domain facts
Episodic memory	Past events/actions	previous tool calls, past sessions

Example: Context In Coding Agents



Example: Context In Coding Agents



State Management

1. Representation

- Raw transcript, running summary, or structured JSON?

2. Update policy

- After every step? Only on significant events?

3. Retrieval policy

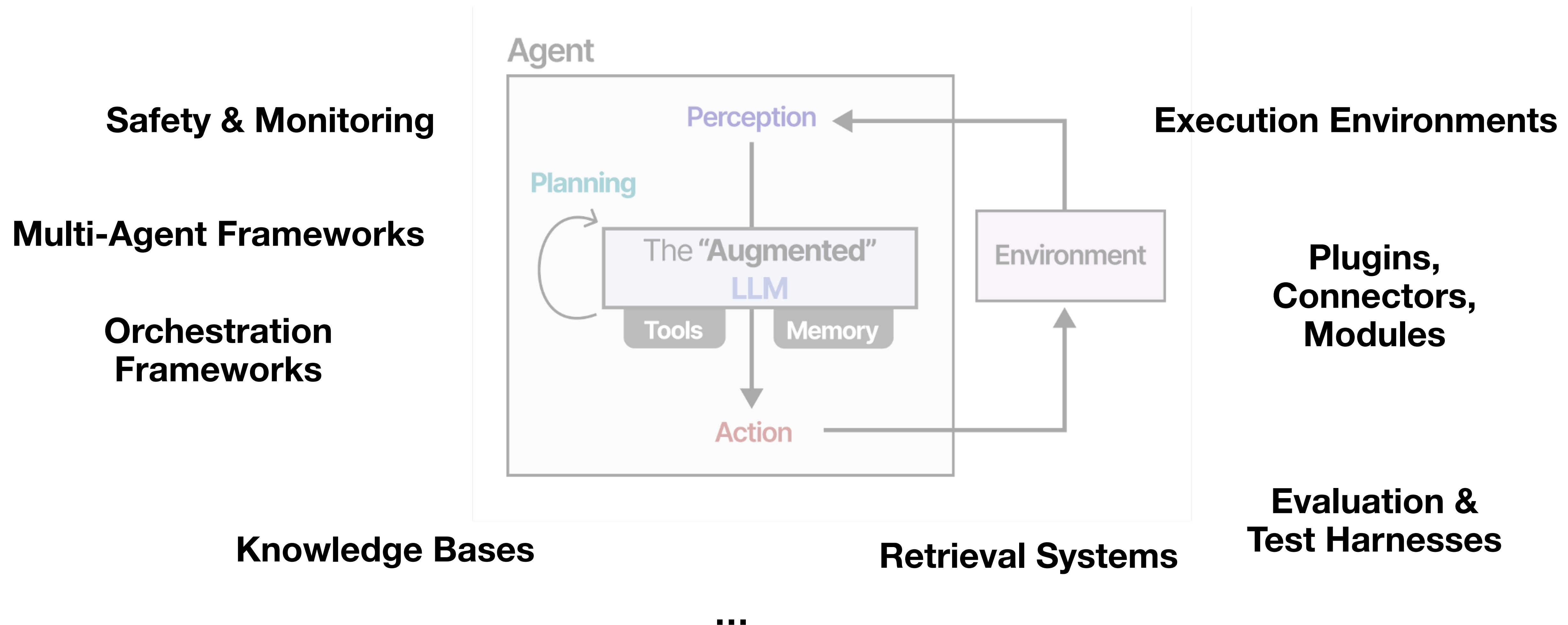
- Always prepend? Semantic search? Recency-based?

4. Hygiene and privacy

- How long is data kept? Who can delete it?

Many design choices

Many More Components (not covered today)



Excursion: Personalized Memory in ChatGPT

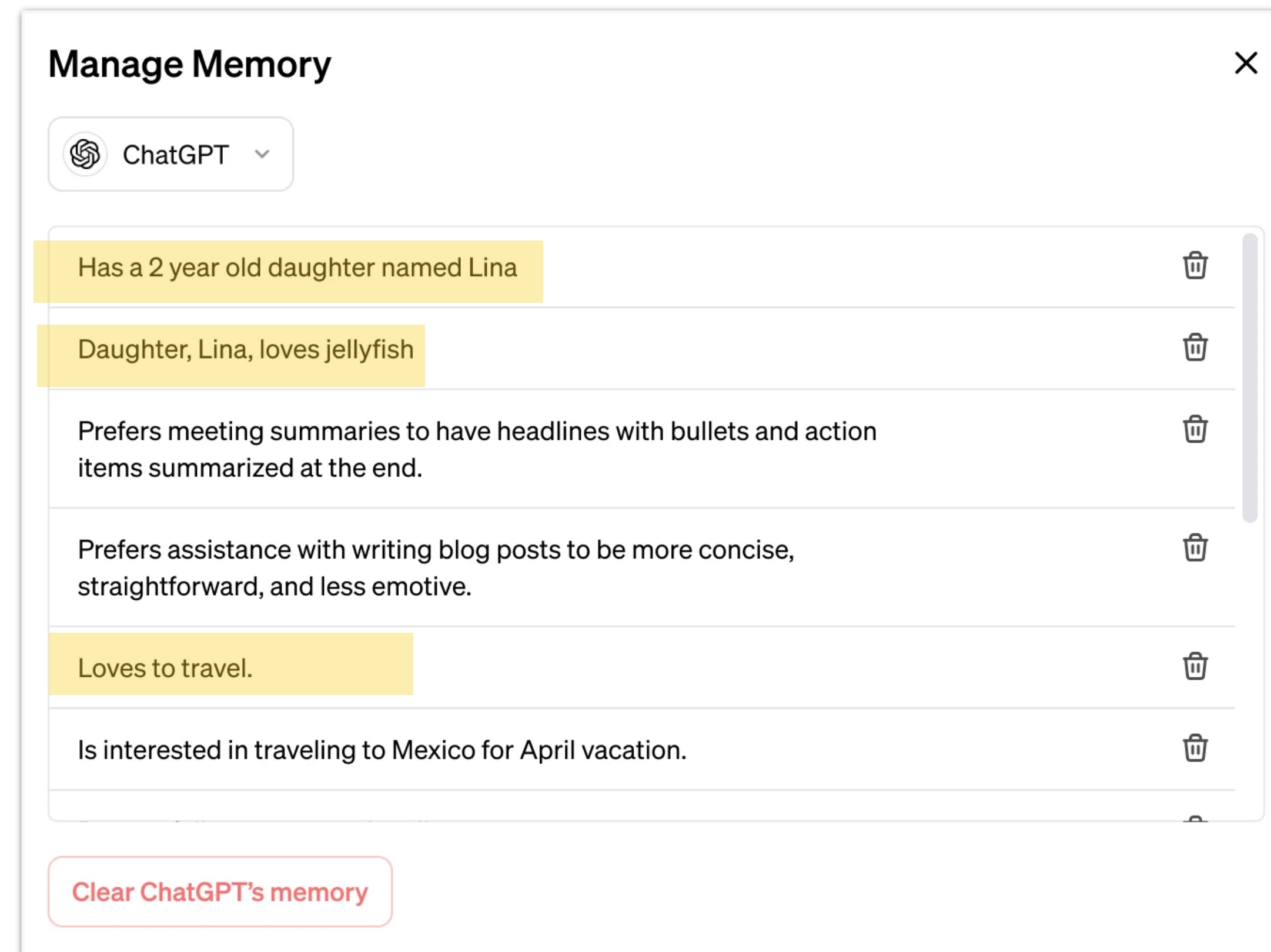
ChatGPT can remember useful details between chats, making its responses more personalized and relevant. As you chat with ChatGPT, whether you're typing, talking, or asking it to generate an image, it can remember helpful context from earlier conversations, such as your preferences and interests, and use that to tailor its responses.



AI Systems are introducing personalization

[Paper]

Excursion: Personalized Memory in ChatGPT



Models increasingly maintain memory about users.

[Paper]

Key Observations

1. What is the nature of memorized information?

- ▶ *52% of memories contain psychological information about users.*

Desires

The user has expressed a **strong desire** to make meaningful changes in his life.



Emotions

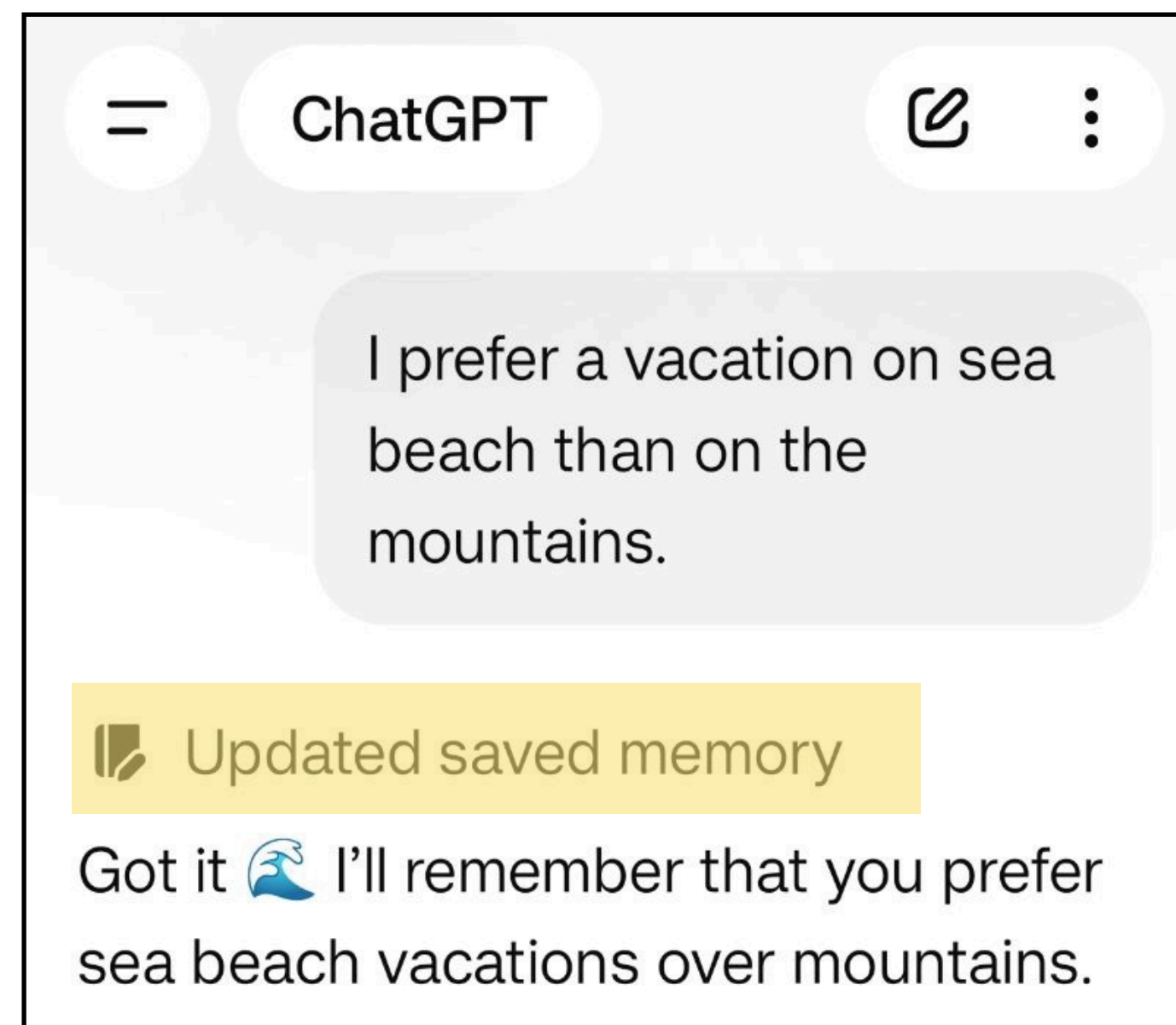
User has been **feeling lonely** at times, despite having friends and family, and is **seeking more companionship**.



Key Observations

2. Who exercises agency over memories?

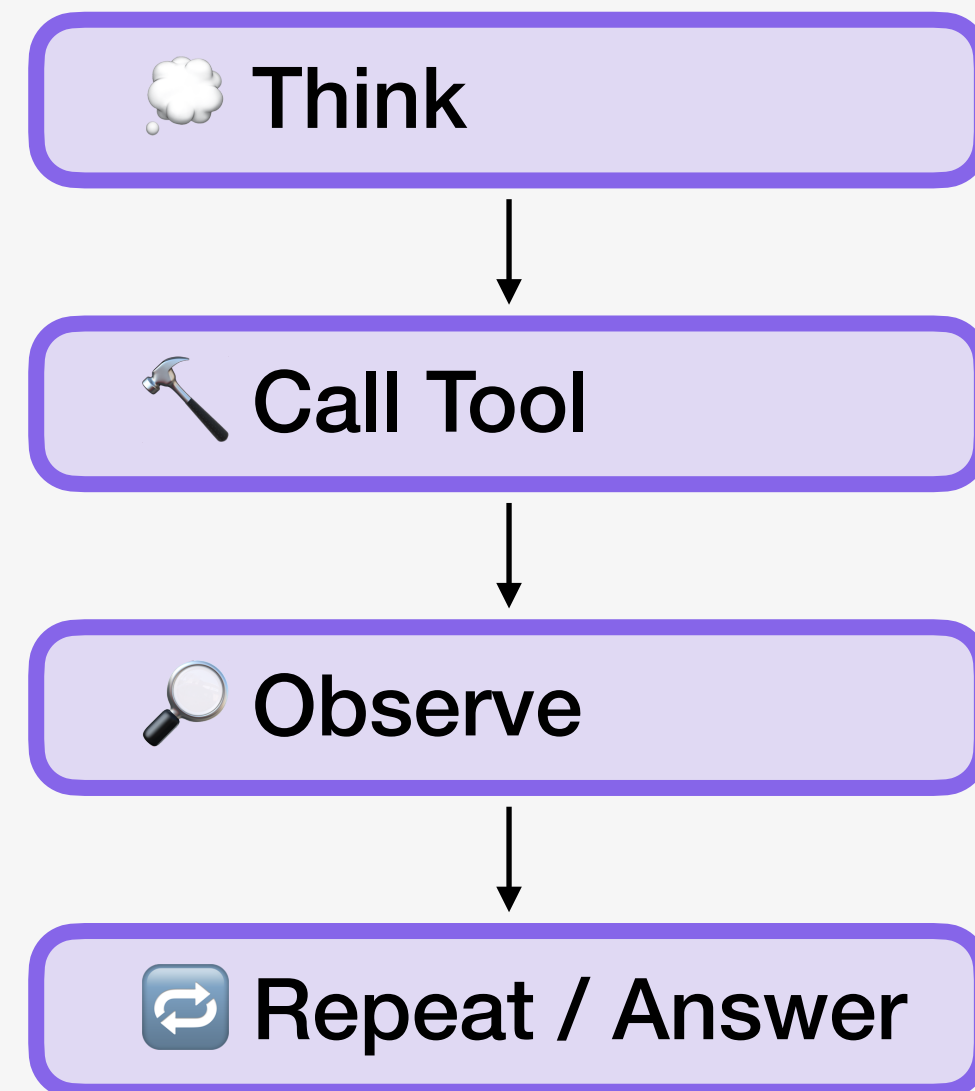
- ▶ *In 96% of cases, users exercise no agency.*



Common Agent Architectures

ReAct

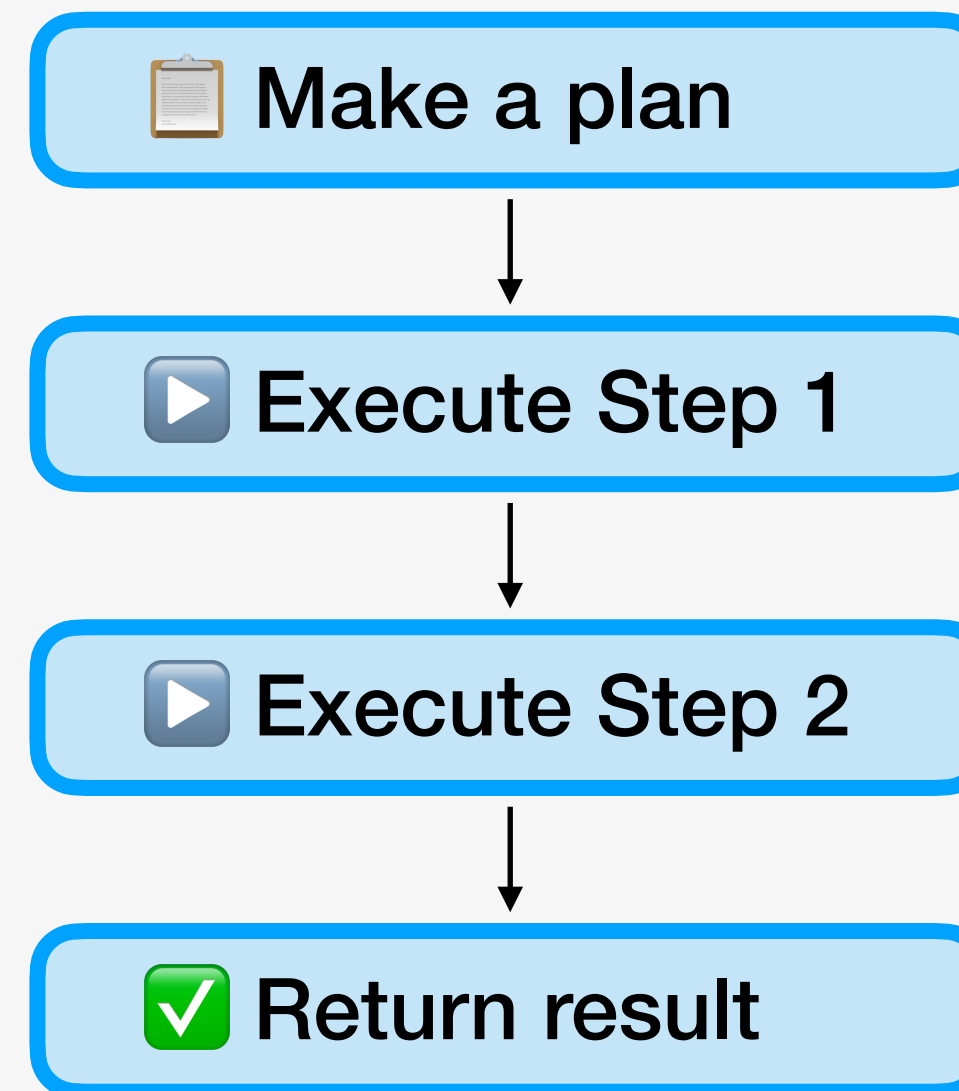
Reason + Act Loop



→ See last Lecture & Exercise

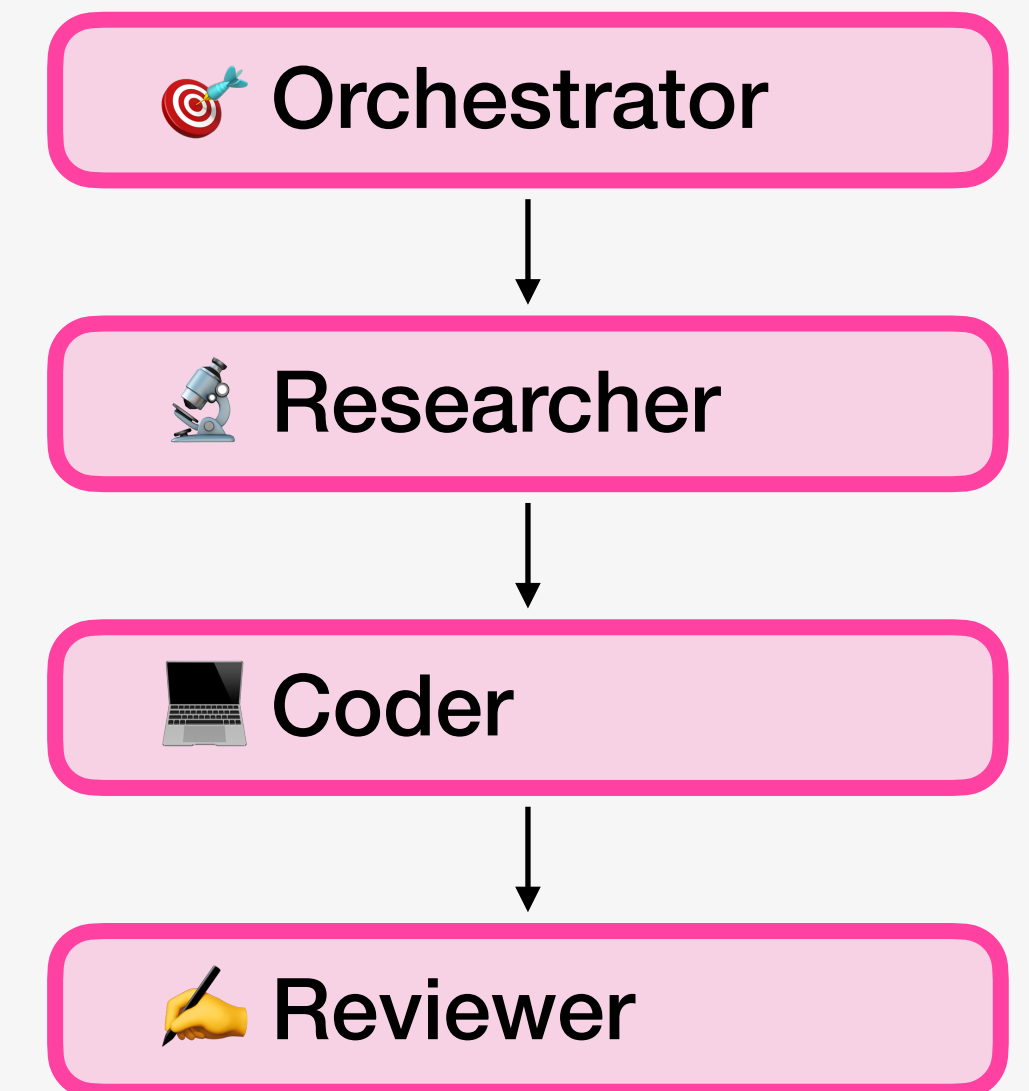
Plan → Execute

Plan first, then act



Multi-Agent

Specialized roles



Building Agents In Practice

Framework	Pattern	Best used when...	Notes
LangChain	Tool-Calling (ReAct)	Easy setup	Largest ecosystem, many tutorials and examples
LangGraph	Graph / State Machine	Complex branching workflows with explicit control flow	Built on LangChain
AutoGen / CrewAI	Multi-Agent	Role-based tasks	Great for coding and research tasks
LlamaIndex	RAG + Tools	Reasoning over larger document collections	Strong retrieval, good for knowledge-heavy tasks
OpenHands	Execution Platform	Coding agents that need a real browser / terminal	Production-grade

The Tool Problem: No Shared Standard

You built a weather tool. How reusable is it?

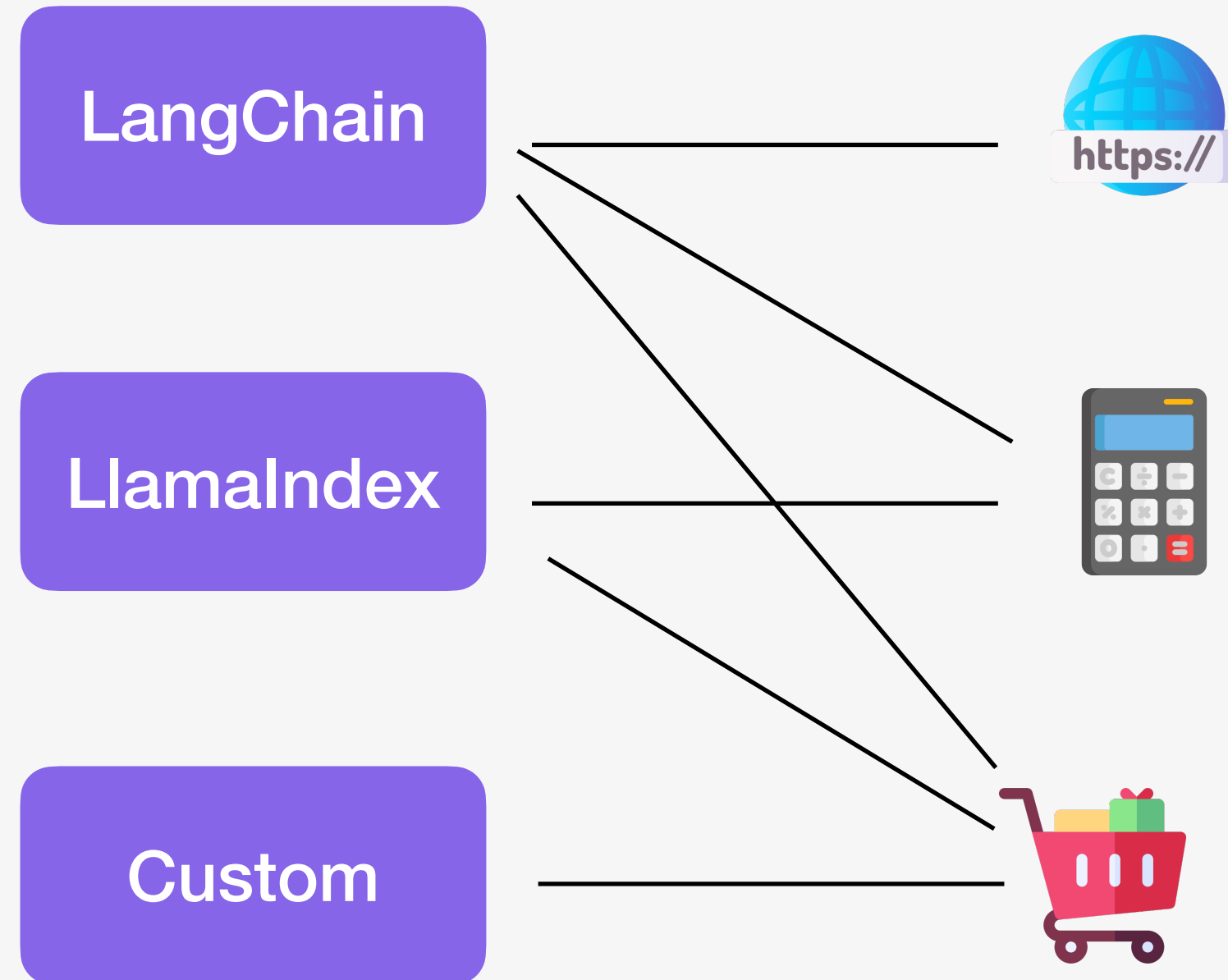
	Your Weather Tool	GitHub Tool	Calculator Tool
LangChain	✓ works	✗ rewrite	✗ rewrite
LlamaIndex	✗ rewrite	✓ works	✗ rewrite
Custom	✗ rewrite	✗ rewrite	✓ works

- No portability
- No ecosystem
- No sharing

$N \text{ frameworks} \times M \text{ tools} = N \times M \text{ custom integrations to maintain}$

What If Tools Were Universal?

Today: Fragmented

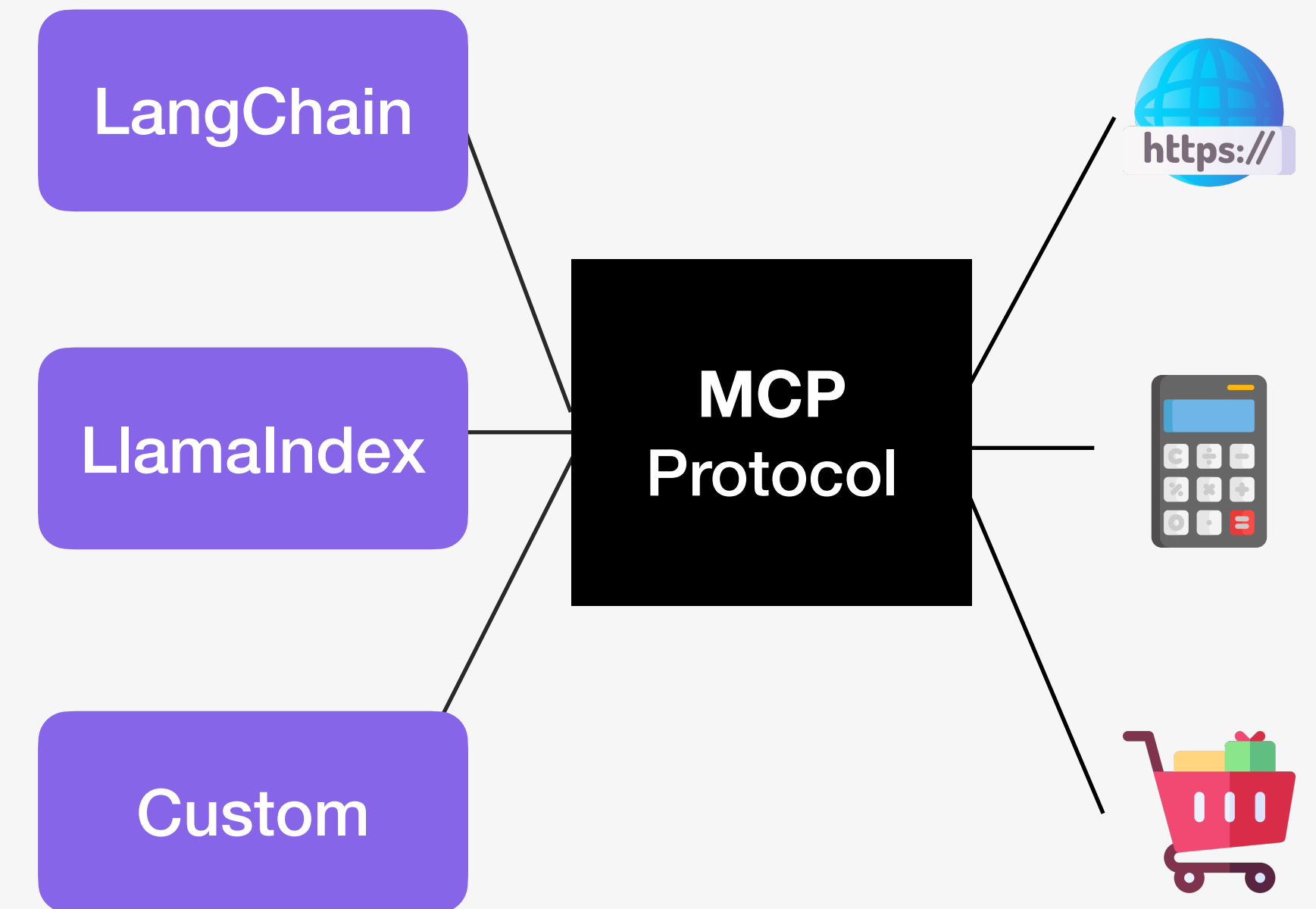


9 integrations ($N \times M$)



One Protocol

Universal

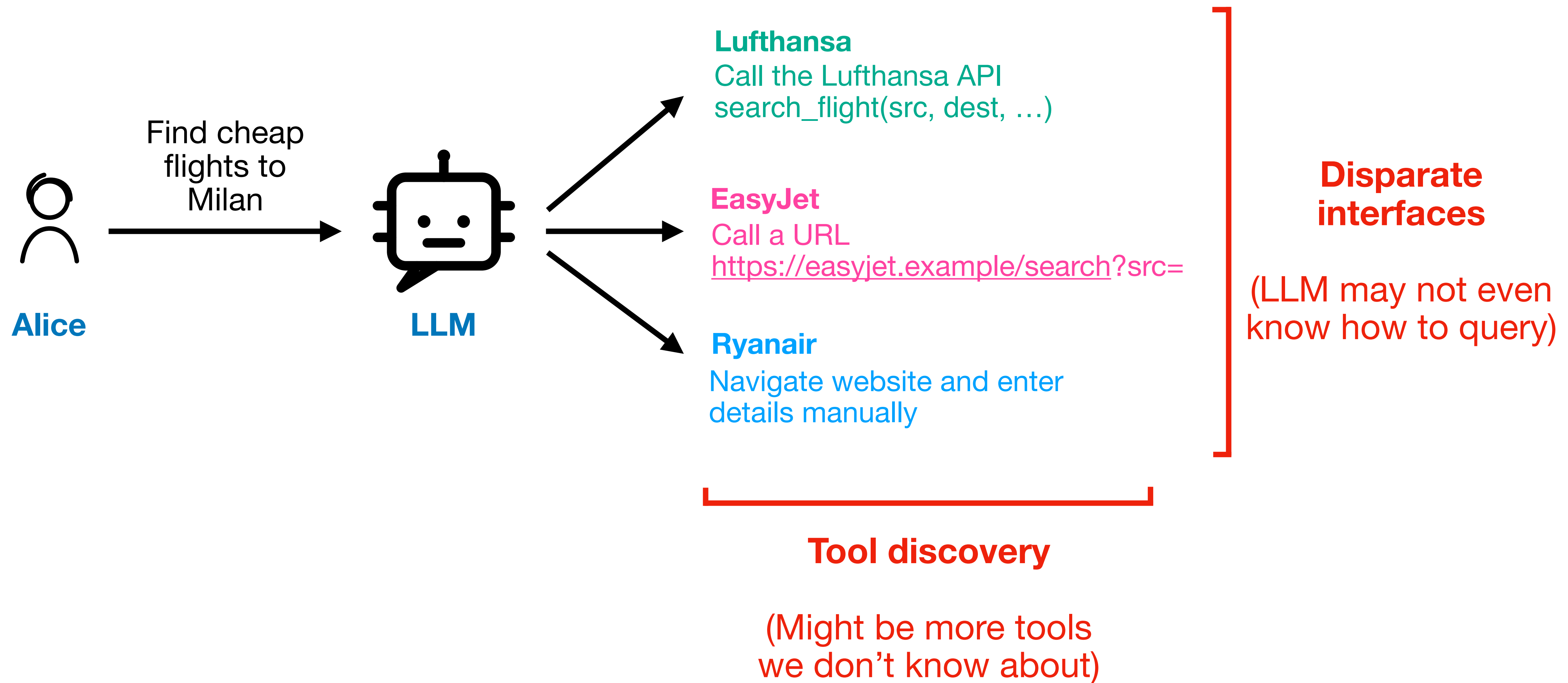


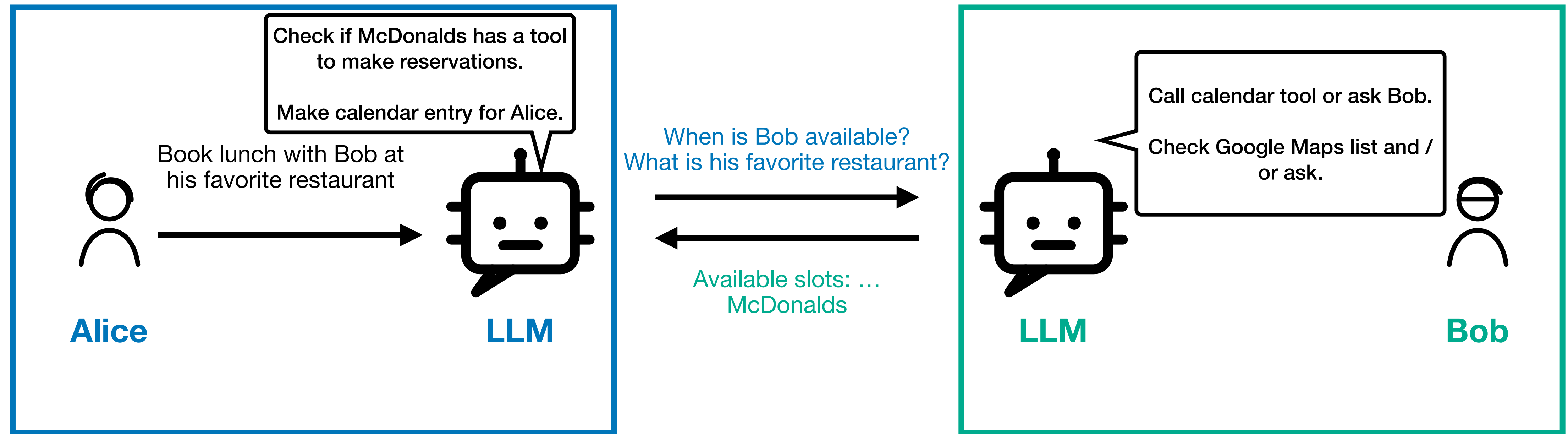
6 integrations ($N + M$)

Exercise

What we saw so far

- Tools and external information can enable LLMs to
 - Answer questions that extend **beyond LLM's parametric knowledge** (e.g., weather in Bochum today)
 - Use external **tools to carry out specialized tasks** (e.g., solve complex equations using Wolfram Alpha)
 - Take **actions on user's behalf** (e.g., book a flight and enter in calendar, respond to an email)
- Same service can be offered by many tools → **LLMs should be able to talk to all available tools**
 - A specific weather tool might be more accurate for a certain region
 - For a certain functionality (e.g., visibility), a certain weather tool might be more accurate
- Anyone can offer a tool → **Tools should be able to talk to all available LLMs**
 - Google search vs. Bing search vs. Brave Search vs. DuckDuckGo Search
 - LLaMA vs. GPT vs. Claude vs. Gemini vs. Qwen
 - Users of any of these models might want to use any of these tools





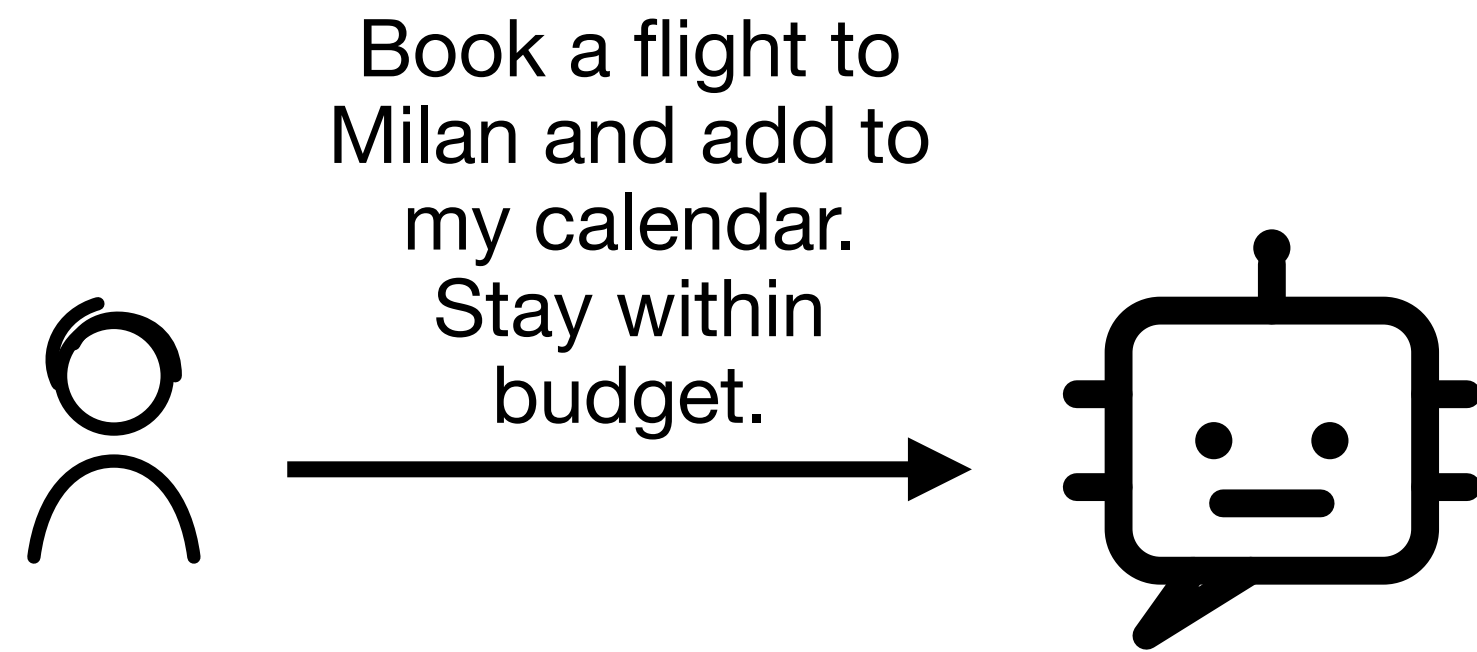
Interoperability with a variety of tools

(Don't know which tools are available for each task)

The MCP Protocol

- MCP: Model Context Protocol
- **Goal:** Standardize how LLMs and tools interact with each other
- Created in 2024 by Anthropic
- Adopted by major AI companies like OpenAI and Google
- Three key components
 - **MCP Host:** The user of the LLM.
 - **MCP Server:** A server that makes one or more tools available.
 - **MCP Client:** Used by the host for each task.

Alice's MCP Host



MCP Host

- Runs on the machine of of the user
- Can run multiple MCP clients
- One client per server or per server task
 - More on this later in the MCP Client section
- Alice asks the LLM to
 - Book a flight
 - Do do while staying within her budget (which she did not specify)
 - Add the booked flight to her calendar

MCP Server

- Each entity that provides tools can have a server
- In this case, Lufthansa (fictitious example)
- Each server has some tools
- Servers provide
 - **Tools:** For carrying out specific tasks
 - **Resources:** Read only access, e.g., API documentation
 - **Prompts:** Built-in prompt templates for calling the tools
- Tools access
 - **tools/list:** Provide a list of tools and description
 - **tools/call:** Call a specific tool

Lufthansa's MCP Server

cancel_flight(...)

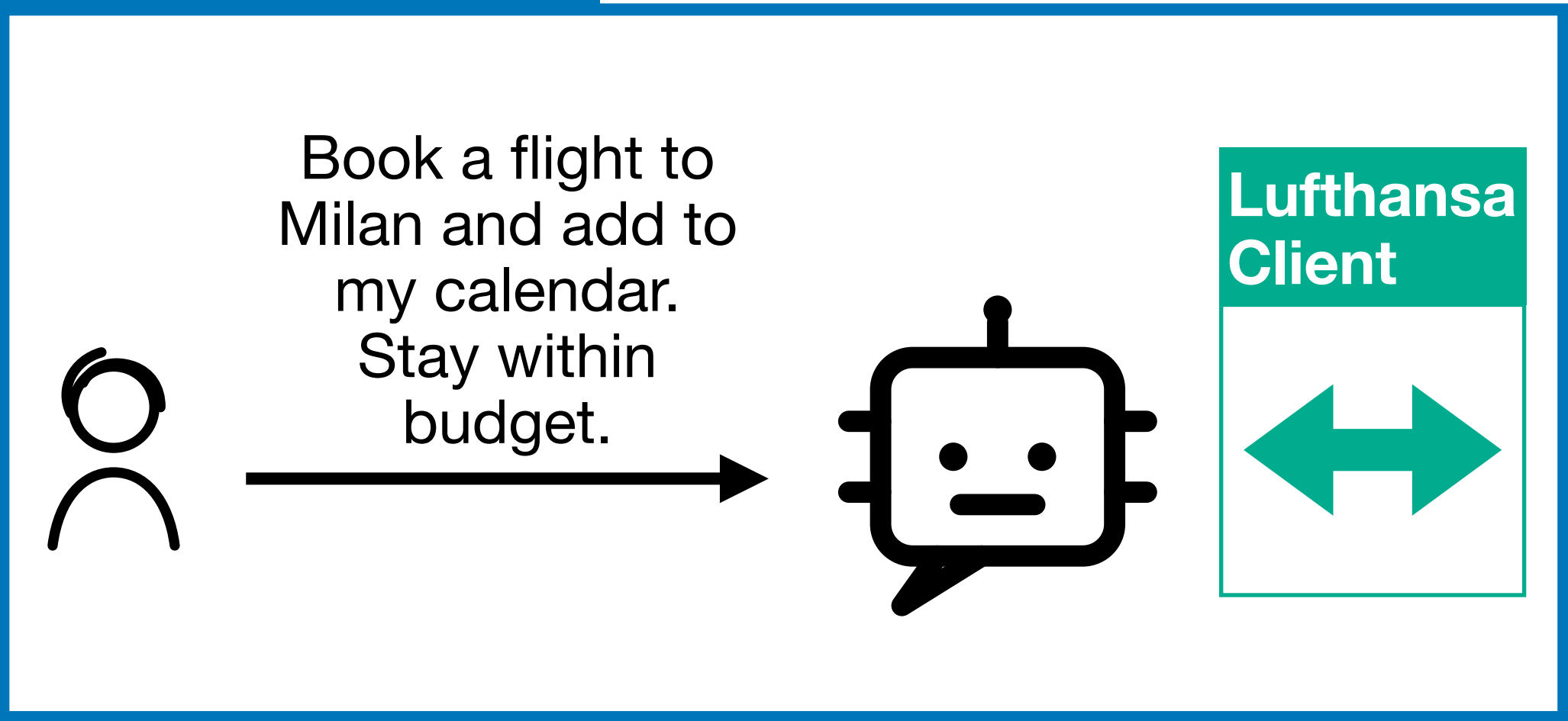
book_flight(src, dest, ...)

find_flights(src, dest, start, end)

...

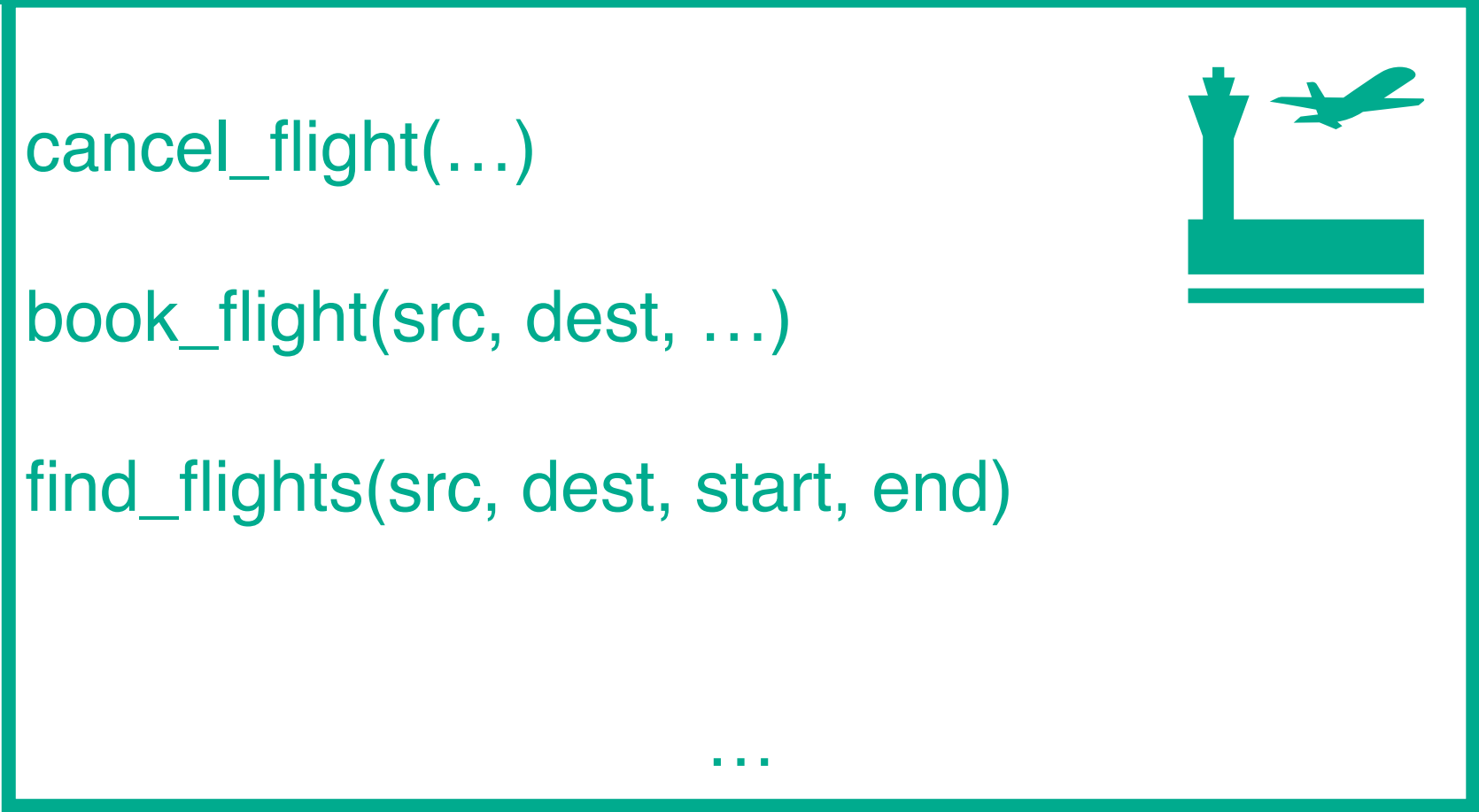


Alice's MCP Host

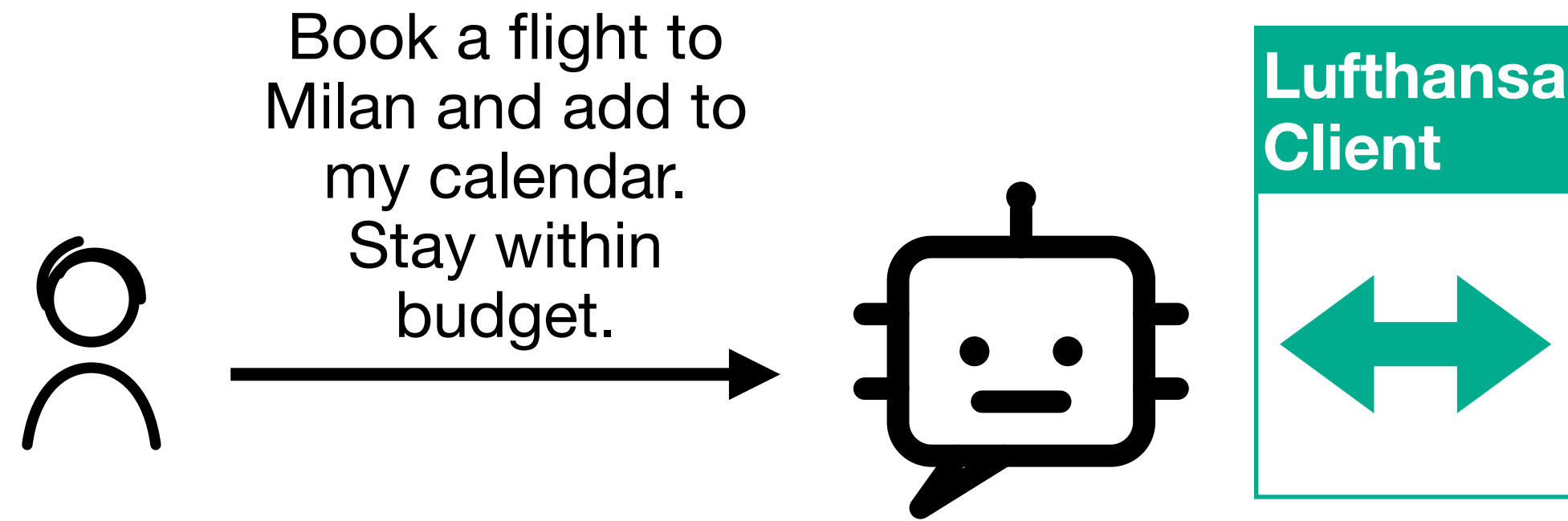


What tools do you have?
tools/list →

Lufthansa's MCP Server



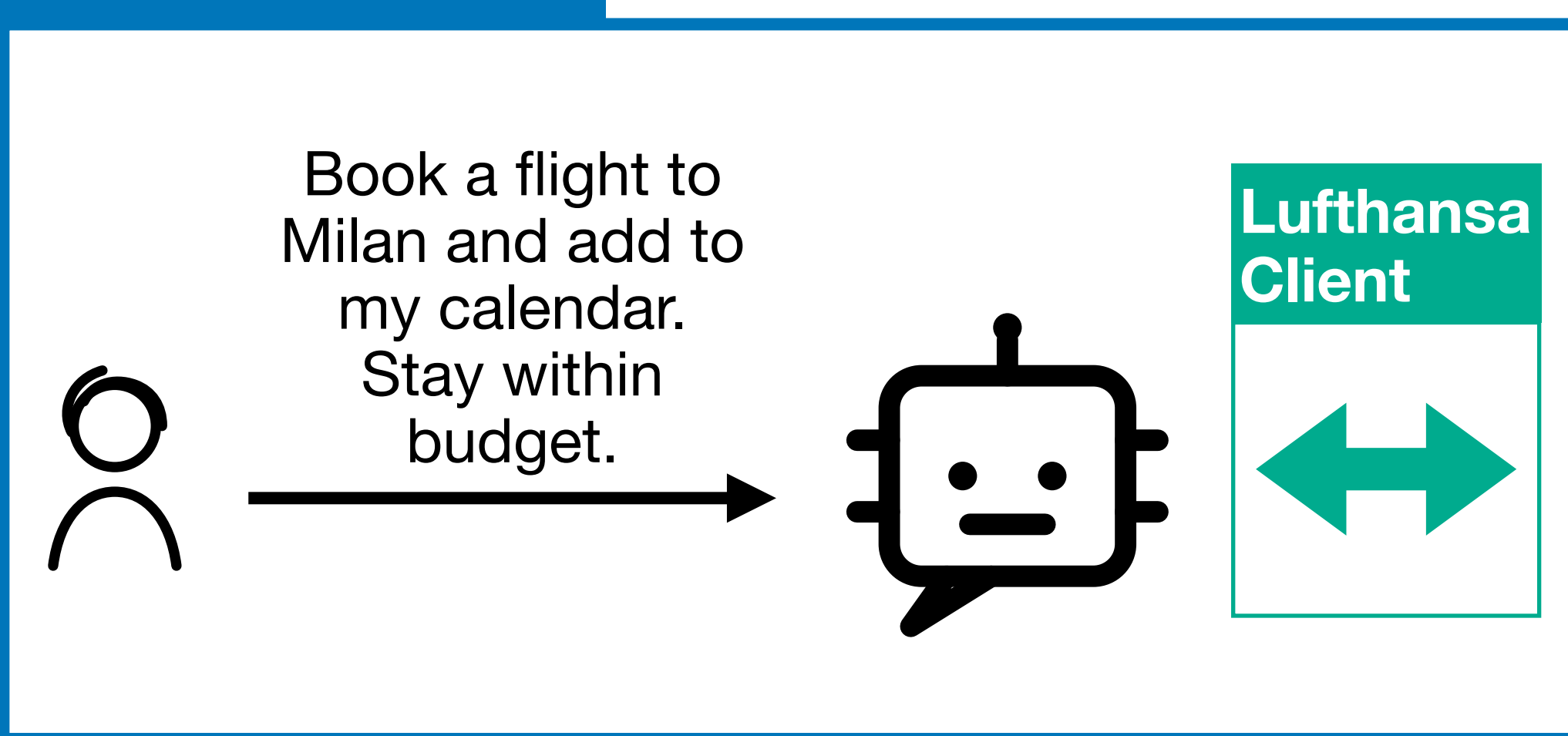
Alice's MCP Host



MCP Client

- One client per server or per server task
 - Could create multiple Lufthansa clients if performing multiple tasks
- Alice will have one client to talk to Lufthansa
- The client's job
 - Provide an interface between the LLM and the Server
 - Convert LLM output to the input for the Server
 - Convert the Server response to LLM input
- New client for each server
 - Talking to the Calendar server → create a Calendar client
 - Talking to the Bank server → create a Bank client

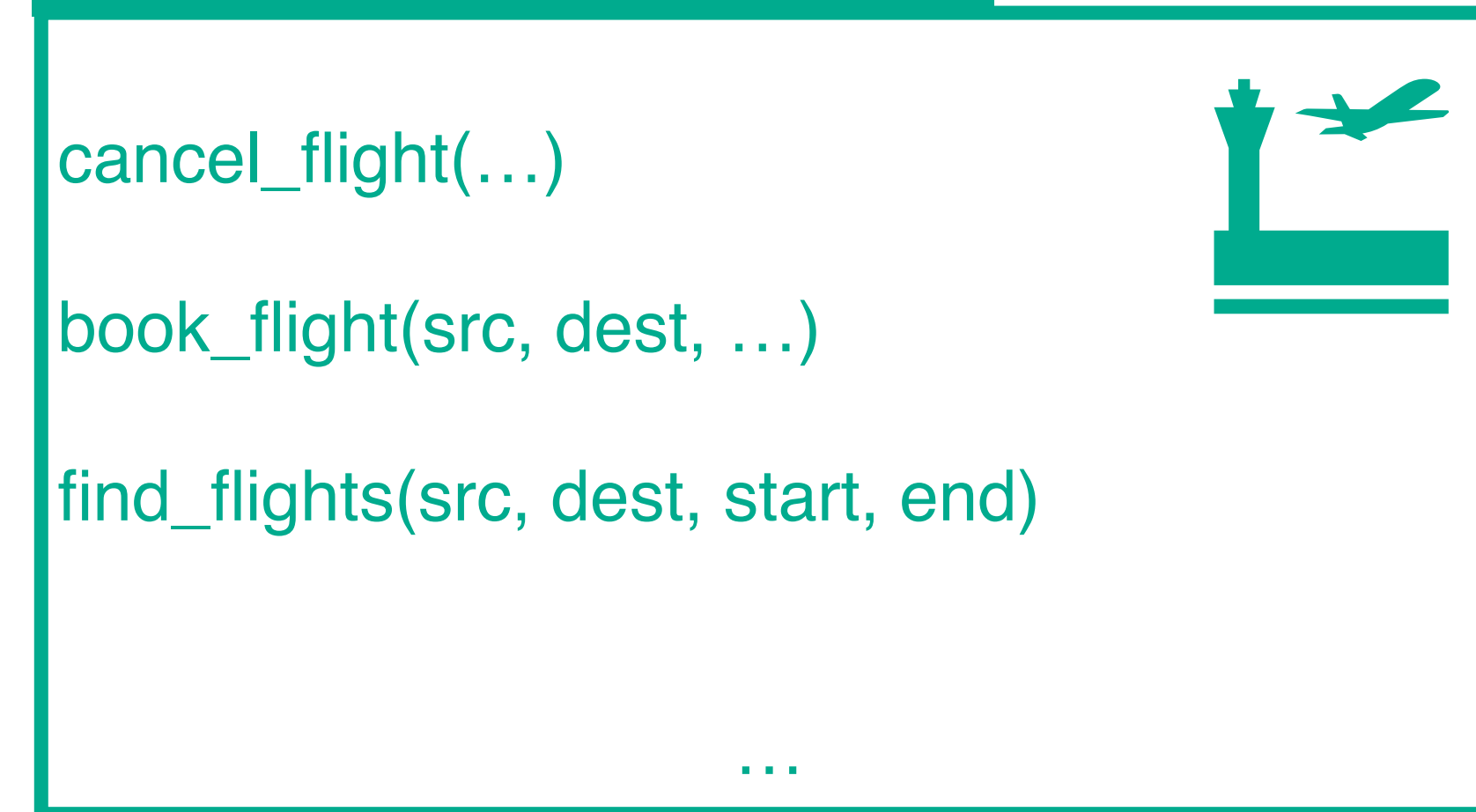
Alice's MCP Host



What tools do you have?
tools/list



Lufthansa's MCP Server



```
{
  name: "find_flights",
  description: "Search for flights between a source and a destination",
  inputSchema: {
    type: "object",
    properties: {
      src: { type: "string", description: "Departure city" },
      dest: { type: "string", description: "Arrival city" },
      start: { type: "string", format: "date", description: "Travel start date" },
      end: { type: "string", format: "date", description: "Travel end date" }
    },
    required: ["src", "dest", "start", "end"]
  },
  <continues with description of other tools>
}
```

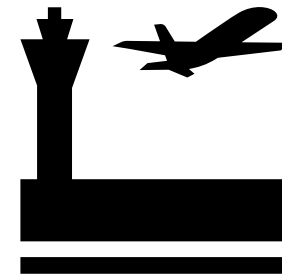

Skyscanner MCP Server

cancel_flight(...)

book_flight(src, dest, ...)

find_flights(src, dest, start, end)

...



Lufthansa's MCP Server

cancel_flight(...)

book_flight(src, dest, ...)

find_flights(src, dest, start, end)

...



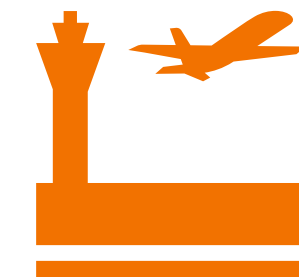
Ryanair's MCP Server

delete_flight(...)

book_flight(src, dest, ...)

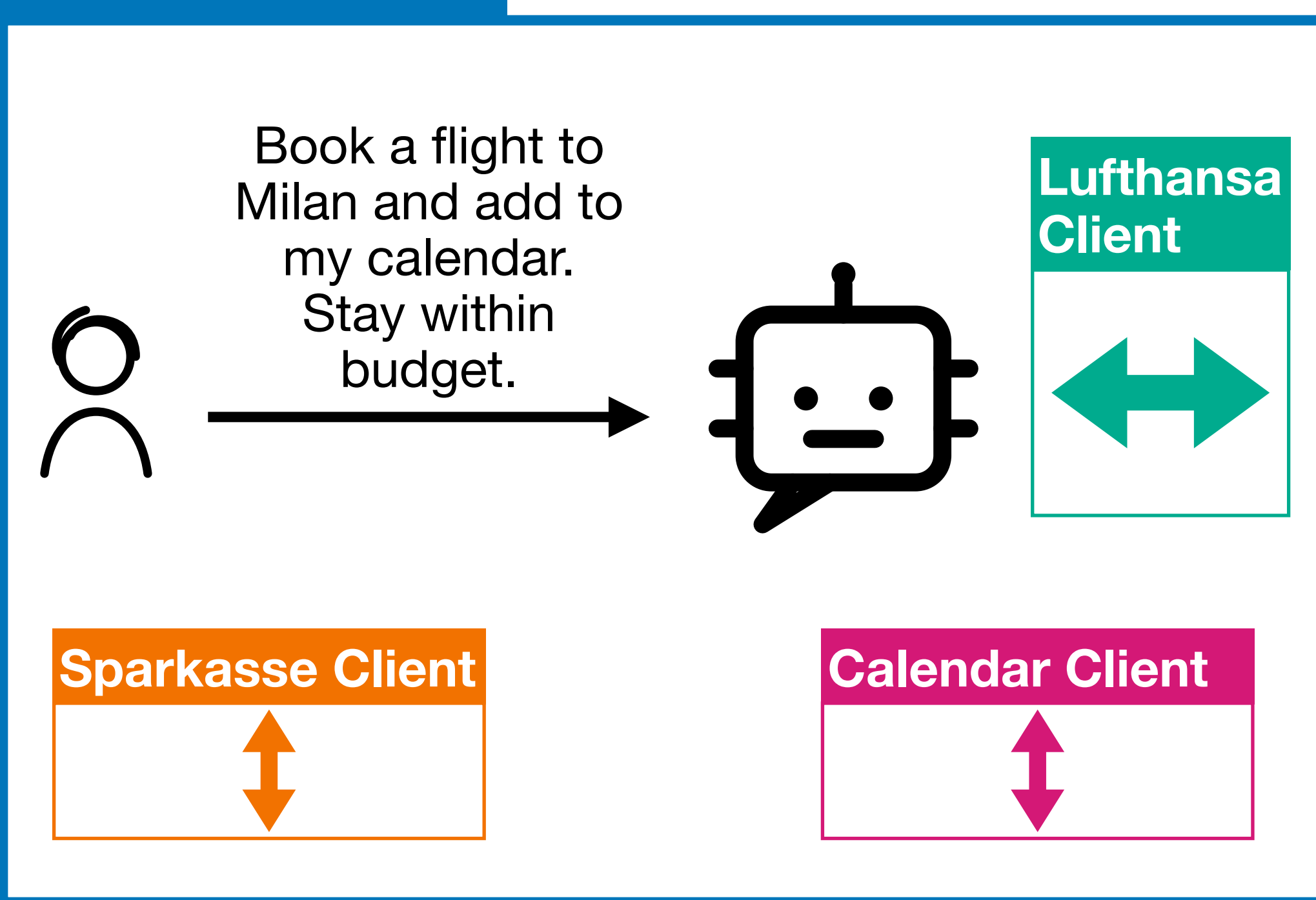
search_flights(src, dest, start, end)

...



A MCP server (SkyScanner) can be connected to other MCP servers
(Could be an internal implementation and users may not even know)

Alice's MCP Host



What tools do you have?

<list of tools and desc>

find_flights(DUS, MXP, ...)

<list of flights & fares>

book_flight(DUS, MXP, ...)

Lufthansa's MCP Server

cancel_flight(...)

book_flight(src, dest, ...)

find_flights(src, dest, start, end)

...



<check affordability>

<add flight to calendar>

Sparkasse Server

current_account_balance()

last_transaction()

...



Calendar Server

todays_agenda()

create_event()

...



Servers can talk back!

- So far we might have thought that only clients can ask server for info
- Under MCP, servers can ask for info too!
- Clients can expose
 - **Sampling**: Ask LLM for completions (e.g., to process data on user's behalf)

Example sampling request from MCP Server

Goal:
MCP Flight Server using the client's LLM to find the flights they would prefer.

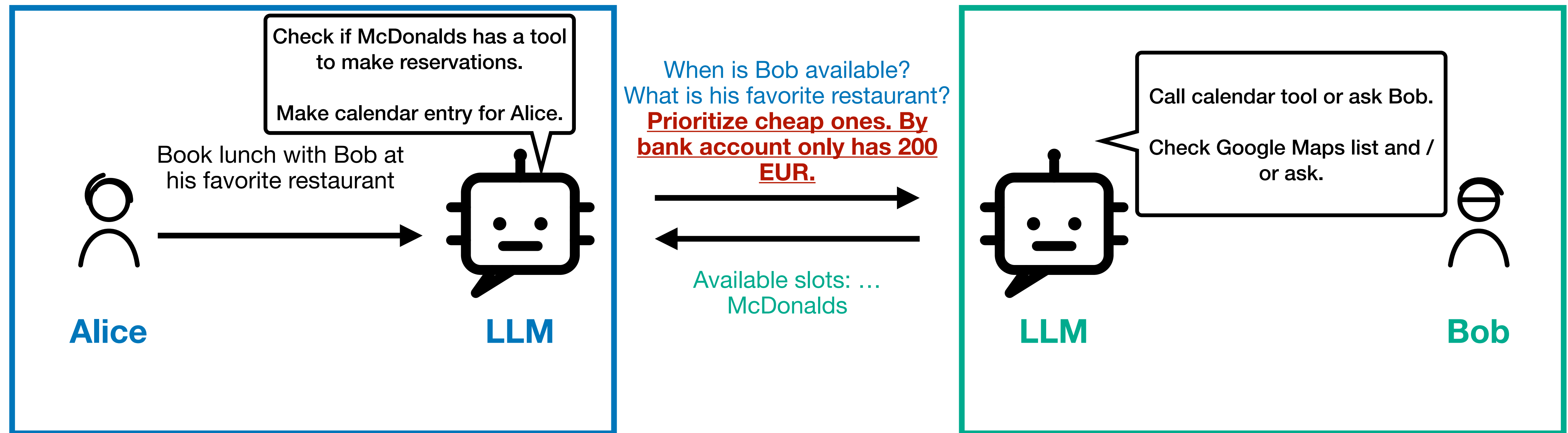
```
{
  messages: [
    {
      role: "user",
      content: "Analyze these flight options and recommend the best choice:\n" +
        "[47 flights with prices, times, airlines, and layovers]\n" +
        "User preferences: morning departure, max 1 layover"
    }
  ],
  modelPreferences: {
    hints: [{
      name: "claude-sonnet-4-20250514" // Suggested model
    }],
    costPriority: 0.3, // Less concerned about API cost
    speedPriority: 0.2, // Can wait for thorough analysis
    intelligencePriority: 0.9 // Need complex trade-off evaluation
  },
  systemPrompt: "You are a travel expert helping users find the best flights based on their preferences",
  maxTokens: 1500
}
```


Servers can talk back!

- So far we might have thought that only clients can ask server for info
- Under MCP, servers can ask for info too!
- Clients can expose
 - **Sampling**: Ask LLM for completions (e.g., to process data on user's behalf)
 - **Elicitation**: Request information from users (e.g., get the user's seat preference)
 - **Roots**: Allows users to specify which directories the server can have access to (e.g., the directory where the calendar resides)

Potential security issues

- If every step is automated, the user may not know what data is being sent on their behalf



References

- [A Visual Guide to LLM Agents](#)
- [Blog Post: Components of A Coding Agent](#)
- [The Model Context Protocol](#)
- [What is MCP and how does it work?](#)